SMART VEHICLE RENTAL SYSTEM

Micro Project Report

Submitted to the APJ Abdul Kalam Technological University, Kerala
in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

KRISHNA PRIYA S
NEETHU S
KRISHNA PRABHA S
SHERIN SUJEESH PANICKER
MEKHA G

Guided by

ANISH A AZIZ
Assistant Professor
Department of Computer Science and Engineering
College of Engineering Perumon

October 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING PERUMON
CAPE - GOVT. OF KERALA, KOLLAM (D), KERALA

TABLE OF CONTENTS

Sl. No.	Contents
1	Introduction
2	Scope of the Micro-Project
3	Technologies Used
4	Problem Statement
5	Literature Review / Background
6	Implementation
7	Details of Java Classes and Architecture
8	Results and Discussion
9	Conclusion
10	References

1. INTRODUCTION

Vehicle rental services give customers temporary access to vehicles for personal, educational, business and travel needs. In many small rental businesses, recording customer details, checking vehicle availability, maintaining bookings and preparing bills are still done manually in registers, which becomes difficult to manage as the number of customers and vehicles grows.

The Smart Vehicle Rental System is a Java-based desktop application that organises these activities through a graphical user interface. It has separate customer and administrator sections. Customers can register, log in, view available vehicles, book a vehicle, choose a payment method and view a bill. Administrators can manage vehicles, bookings and customer records.

The application uses Java Swing for the interface and ArrayList collections for temporary storage. It demonstrates classes, objects, encapsulation, constructors, event handling and exception handling. The current version is an academic prototype: data is kept in memory and is lost when the program closes, and payment is only simulated.

2. SCOPE OF THE MICRO-PROJECT

The scope is to develop a simple desktop application that demonstrates the main stages of a vehicle rental process and the flow of information between application screens.

2.1 Customer functionalities

- Registration with name, email, phone number and password, followed by login.
- Viewing and updating the profile; viewing available vehicles and their details.
- Booking a vehicle for a number of days and selecting a payment method (UPI, Card or Cash).
- Viewing the generated bill, current bookings and rental history.

2.2 Administrator functionalities

- Separate administrator login and dashboard.
- Adding, deleting and refreshing vehicle records.
- Viewing and cancelling bookings; viewing and deleting customer records.

2.3 Project boundaries

The application runs on a local Java desktop. It uses no database, cloud server or real payment gateway, and the administrator credentials are hard-coded for demonstration.

3. TECHNOLOGIES USED

1. **Java:** core programming language used for the application logic and classes.

2. **Java Swing:** used to build windows, labels, text fields, buttons, tables, combo boxes and dialog boxes.

3. **AWT:** provides layout managers (BorderLayout, FlowLayout, GridLayout, GridBagLayout) and event support.

4. **ArrayList:** stores customer, vehicle and booking objects temporarily in memory.

5. **Eclipse IDE:** used for writing, compiling and testing the Java source files.

6. **Git and GitHub:** used for version control and collaboration among team members.

4. PROBLEM STATEMENT

In a small rental business, maintaining vehicle availability, customer details and booking records manually leads to difficulties. Staff may need to search through registers to confirm whether a vehicle is free, and repeated manual entry can cause incomplete records, calculation mistakes and confusion when reviewing previous bookings.

The proposed system brings these activities into one application with separate screens for registration, login, vehicle browsing, booking, payment selection, billing and administrative management.

Objective: to design and implement a beginner-level Java Swing application that organises basic vehicle rental operations, demonstrates object-oriented programming concepts and provides a clear graphical workflow for customers and administrators.

5. LITERATURE REVIEW / BACKGROUND

A rental process generally involves registering a customer, checking availability, selecting a vehicle, deciding the rental period, calculating the charge, recording payment and keeping booking history. A register-based process can do this, but retrieving and updating information becomes slow as records grow.

Java is an object-oriented language suited to such desktop systems. Java Swing offers reusable GUI components, layout managers arrange them within windows, and event listeners let the application respond to actions such as button clicks.

The project uses a simple object-oriented structure: Customer, Vehicle and Booking are separate classes, and AppData holds the shared in-memory collections. This suits a small prototype, although a production system would need persistent storage, stronger authentication, validation and secure payment integration.

6. IMPLEMENTATION

Each major screen is implemented as a Java class extending JFrame, and buttons are connected to event-handling code. The system consists of the following modules.

6.1 Customer modules

1. Login and Registration: LoginFrame validates the customer login and links to CustomerRegistrationFrame, which creates a Customer object from the entered details.

2. Dashboard and Profile: CustomerDashboardFrame is the main menu leading to the profile, available vehicles, bookings and rental history.

3. Vehicle Browsing: AvailableVehiclesFrame lists vehicles marked available and VehicleDetailsFrame shows the selected vehicle.

4. Booking: BookingFrame accepts the number of days, validates it and calculates the rental amount.

5. Payment and Billing: PaymentFrame simulates UPI, Card or Cash payment and BillingFrame shows the bill.

6. Bookings and History: MyBookingsFrame and RentalHistoryFrame display the customer's records for the current session.

6.2 Administrator modules

1. Admin Login and Dashboard: AdminLoginFrame uses hard-coded demonstration credentials; AdminDashboardFrame links to the management screens.

2. Vehicle Management: VehicleManagementFrame adds, deletes and refreshes vehicle records.

3. Booking Management: BookingManagementFrame shows bookings and cancels them; a cancelled booking makes its vehicle available again.

4. Customer Management: CustomerManagementFrame shows customer records and can delete a customer.

6.3 Booking and amount calculation

When a customer selects a vehicle and enters the rental duration, the application calculates the rental amount as the daily rent multiplied by the number of days:

Total rental amount = Rent per day × Number of rental days

For example, a vehicle with a daily rent of 1500 booked for 3 days gives a total of 4500. Text input is converted to numeric values, and a NumberFormatException is caught if the input is invalid. After booking, the vehicle is marked unavailable so that it no longer appears in the customer's vehicle list.

6.4 Event handling and exception handling

Action events respond to button clicks. ActionListener implementations, including lambda expressions, connect buttons to operations such as opening another frame, creating a booking, confirming a simulated payment or cancelling a booking. Try-catch blocks are used for input conversions so that invalid numeric input does not terminate the program.

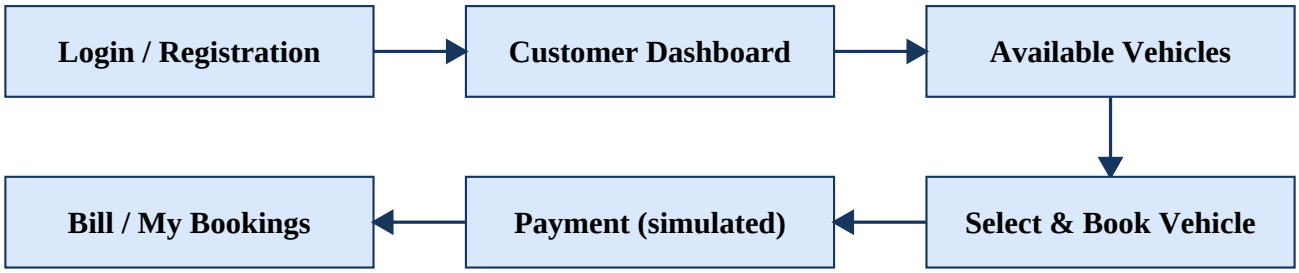
6.5 Data storage

The AppData class keeps static ArrayList collections of customers, vehicles and bookings, along with a reference to the logged-in customer. Sample vehicles are created when the application starts. Because the collections exist only in memory, all changes are lost when the application is closed, and no database is connected in this version.

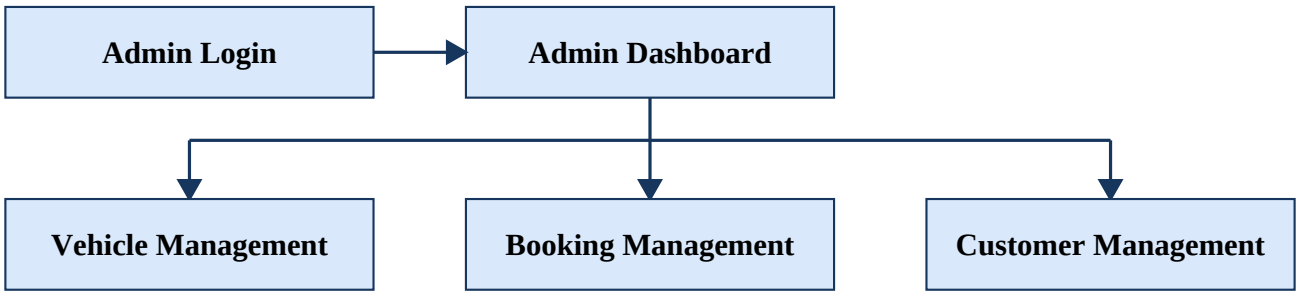
6.6 Application workflow

The diagram below summarises how a customer and an administrator move through the screens of the application.

Customer workflow



Administrator workflow



6.7 Application screenshots

The four main screens of the application are shown below.

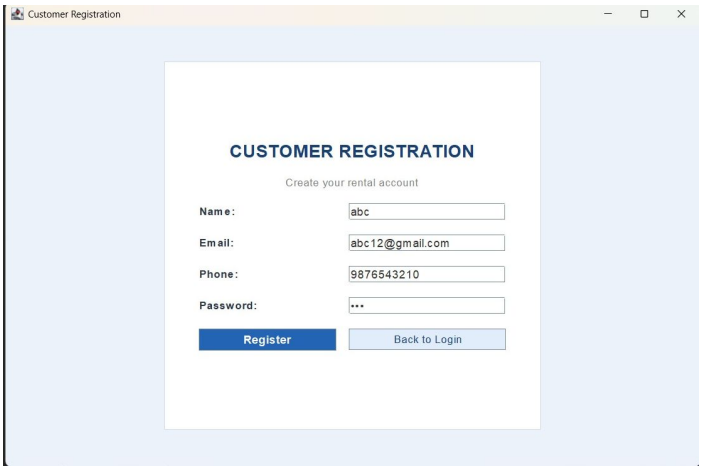


Fig. 1: Customer Registration

Registration form for name, email, phone number and password.

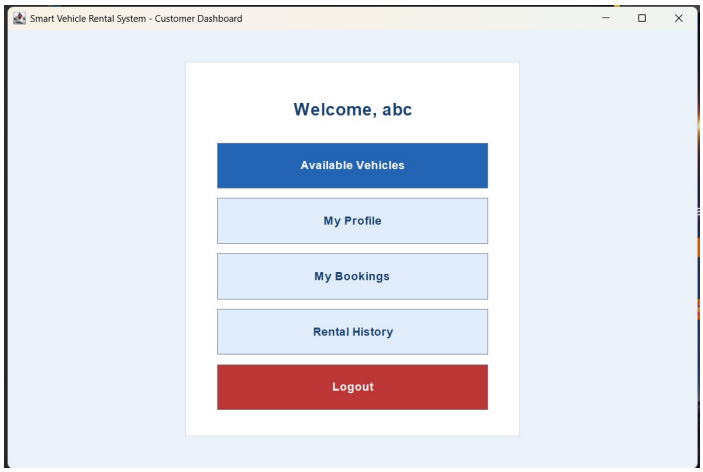


Fig. 2: Customer Dashboard

Main menu with vehicles, profile, bookings, rental history and logout.

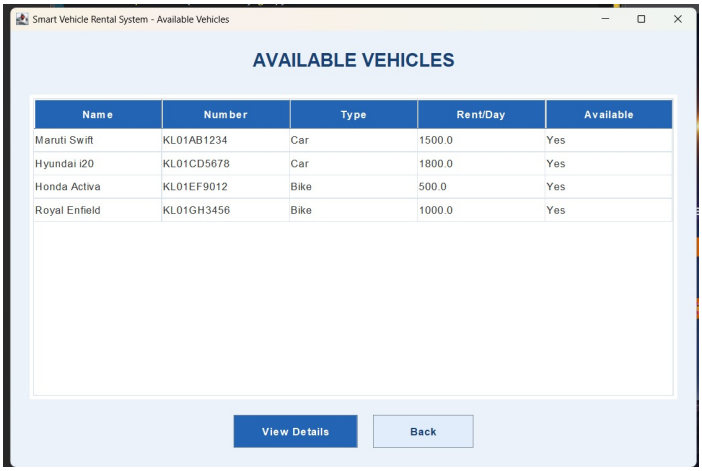


Fig. 3: Available Vehicles

Vehicles currently free for booking, with rent per day.

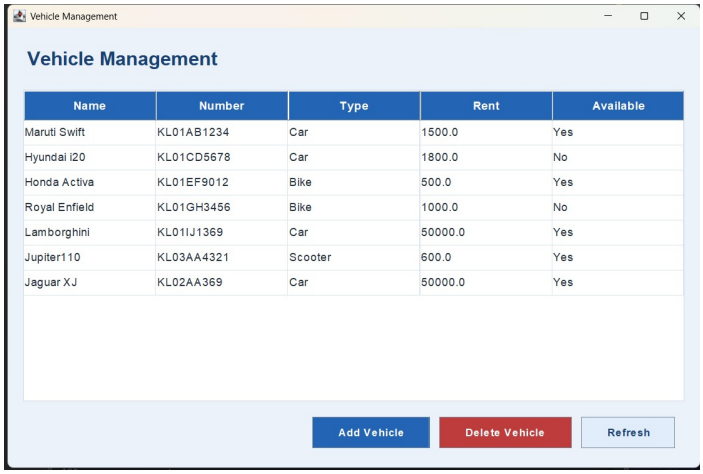


Fig. 4: Vehicle Management (Admin)

Vehicle table with Add, Delete and Refresh buttons.

7. DETAILS OF JAVA CLASSES AND ARCHITECTURE

The project contains 16 GUI frame classes and four supporting classes, making 20 Java classes in total.

7.1 GUI frame classes

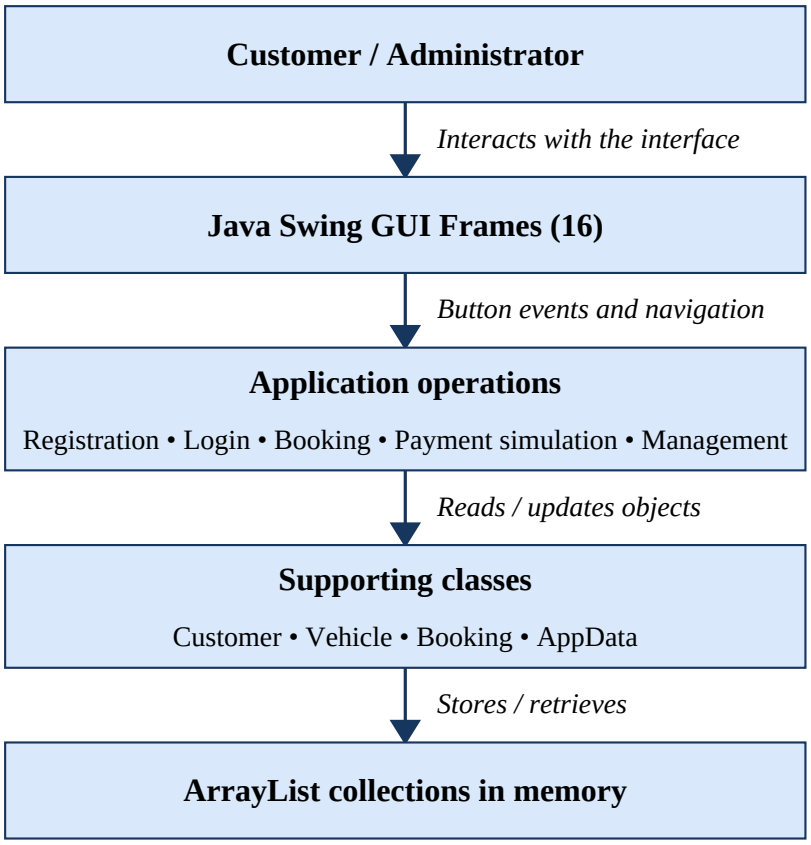
No.	Java class	Main responsibility
1	LoginFrame	Customer login; links to registration / admin login
2	CustomerRegistrationFrame	Collects registration details
3	CustomerDashboardFrame	Main customer navigation screen
4	CustomerProfileFrame	Displays profile information
5	AvailableVehiclesFrame	Lists available vehicles
6	VehicleDetailsFrame	Shows details of a selected vehicle
7	BookingFrame	Accepts rental days, calculates amount
8	PaymentFrame	Simulates payment selection
9	BillingFrame	Displays the rental bill
10	MyBookingsFrame	Displays current bookings
11	RentalHistoryFrame	Displays rental history
12	AdminLoginFrame	Administrator authentication
13	AdminDashboardFrame	Administrator navigation screen
14	VehicleManagementFrame	Adds and deletes vehicle records
15	BookingManagementFrame	Views and cancels bookings
16	CustomerManagementFrame	Views and deletes customer records

7.2 Supporting classes

- **Vehicle:** name, registration number, rent and availability, with getters and setters.
- **Customer:** name, email, phone number and password.
- **Booking:** customer, vehicle, rental days, amount and payment status.
- **AppData:** static ArrayLists of customers, vehicles and bookings, and the logged-in customer.

7.3 Application architecture

The Swing frames form the presentation layer. Event-handling code in the frames performs the operations, the Customer, Vehicle and Booking classes represent the data, and AppData provides the shared in-memory collections.



7.4 Object-oriented concepts used

- **Classes and objects:** Customer, Vehicle and Booking objects represent individual records.
- **Encapsulation:** fields are grouped in classes and accessed through getters and setters.
- **Constructors and methods:** constructors set initial values; methods organise operations.
- **Collections:** ArrayList stores multiple objects of the same type.

8. RESULTS AND DISCUSSION

The Smart Vehicle Rental System was implemented as a Java Swing desktop application. The customer and administrator workflows were tested through the GUI screens, and the application successfully demonstrates navigation between screens and the basic rental sequence using sample and session-based data.

8.1 Functional testing

Test activity	Observed result
Customer registration	Details accepted; customer added to the collection
Customer login	Valid details open the customer dashboard
Vehicle listing	Only available vehicles are displayed
Booking duration	Amount calculated from daily rent and days
Invalid numeric input	Handled by exception handling
Payment and billing	Simulated payment completes; bill is displayed
Vehicle management	Vehicles can be added and deleted
Booking cancellation	Booking cancelled; vehicle available again
Customer management	Customers can be viewed and deleted

8.2 Limitations

- Data is stored only in memory and is lost when the program closes.
- Administrator credentials are hard-coded and payment is simulated.
- There is no database, online service or remote access.

8.3 Future enhancements

- Store data permanently in a database such as MySQL.
- Add secure authentication with password hashing and a real payment gateway.
- Add booking dates, return-date validation and printable invoices.

8.4 Version control and team contribution

The project was developed collaboratively on GitHub. The Insights page below shows the weekly commits and the contribution of each of the five team members.



Fig. 5: GitHub contribution graph of the team

9. CONCLUSION

The Smart Vehicle Rental System demonstrates a basic vehicle rental application using Java and Swing. It provides customer functions such as registration, login, vehicle browsing, booking, payment-method selection, bill viewing and booking history, and administrator functions for managing vehicles, bookings and customers.

The project applied object-oriented concepts, Java collections, GUI components, layout managers, event handling and exception handling in a practical application. The current version is an academic prototype with in-memory storage and simulated payment; adding a database, secure authentication and a real payment service would be the next steps towards a production-ready system.

GitHub path

https://github.com/mekhag-2050/-oop--miniproject-/tree/main/SmartVehicleRental/src/rental/system

REFERENCES

1. Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill Education.
2. Oracle, Java Documentation. https://docs.oracle.com/en/java/
3. Oracle, Creating a GUI With Swing. https://docs.oracle.com/javase/tutorial/uiswing/
4. Oracle, Collections Framework. https://docs.oracle.com/javase/tutorial/collections/
5. GitHub Docs, About GitHub and version control. https://docs.github.com/
6. KTU course materials and laboratory reference materials for Object-Oriented Programming.
7. GitHub, Smart Vehicle Rental System – Project Source Code. https://github.com/mekhag-2050/-oop--miniproject-/tree/main/SmartVehicleRental/src/rental/system